18 岛、睡眠与数据布局

> 目标：解释为什么物体会“搬家”、为什么睡眠必须考虑连通关系，以及这些机制如何减少真正进入热循环的数据。
>
> 本章对应 Box2D 3.2.0，提交 `dc67c40ba2ce1ad384b0c985dbc1b539cba9221b`。 字节量与操作次数是明确假设下的理论估算，不代表本次执行过性能测试。

18.1 从一份简单的 Body 数组开始

第一版引擎完全可以让一个 `Body` 结构包含全部信息：

- 用户数据、名字和形状列表。
- 世界位置、旋转和质量。
- 速度、外力和本步增量。
- 接触、关节和睡眠标志。

每帧遍历数组，对醒着的物体积分，对所有约束顺序求解。 这容易实现，也容易验证。

问题出现在“世界很大，但大多数物体不动”的场景。 假设世界有 100,000 个刚体，只有 1,000 个醒着。 如果每次热循环都扫描整个世界并判断 `isAwake`，仍要读取 100,000 个标志。 增加子步和阶段以后，这项扫描会重复出现。

教学伪代码，非源码：

```
for body in allBodies:
    if body.awake:
        integrate(body)
```

这里并不是判断语句写错了，而是数据组织迫使求解器不断经过不需要的数据。 优化目标应是让热循环天然只看见活跃物体。

18.2 冷热拆分到底节省什么

当前实现把刚体信息拆成三层。

结构	主要用途	典型内容
`b2Body`	身份、API、连接关系	ID、代数、用户数据、邻接索引、睡眠计时
`b2BodySim`	碰撞准备与积分参数	世界变换、质心、力、逆质量、阻尼、尺度
`b2BodyState`	约束热循环反复读写	速度、增量位移、增量旋转、标志

`b2BodySim` 没有保存线速度和角速度。 速度在 `b2BodyState` 中；这直接影响后面的睡眠与唤醒语义。

`b2BodyState` 的布局为：

字段	字节
`linearVelocity`	8
`angularVelocity`	4
`flags`	4
`deltaPosition`	8
`deltaRotation`	8
总计	32

32 B 不只是注释中的愿望，接触求解器使用静态断言检查这一布局。 其他结构的实际大小受 ABI、编译选项和双精度世界模式影响，不应随意写死。

考虑一个明确标注为假设的成本模型。 假设未拆分的一体结构占 256 B，但约束循环实际只需要 32 B。

$$100000 \times 256 = 25{,}600{,}000\ \mathrm{B},$$

$$100000 \times 32 = 3{,}200{,}000\ \mathrm{B}.$$

若只让 1,000 个醒体进入热数组，则有效状态范围变为：

$$1000 \times 32 = 32{,}000\ \mathrm{B}.$$

这些是数据跨度与有效工作集的比较，不是自动得到八倍或八百倍加速。 实际时间还受缓存命中、访问模式、计算量和预取影响。

若假设缓存行为 64 B，一行可容纳两个连续的 `b2BodyState`。 这改善局部性，但也意味着两个线程写不同刚体时仍可能发生伪共享。 数据拆分不会自动解决所有并发访存问题。

实验：记录热循环迭代数、`bodyStates.count` 与 `capacity`。 再测实际时间，检查主要收益来自少扫描物体，还是更紧凑的访存。

18.3 稳定 ID 不等于稳定位置

让用户直接保存数组元素地址，会使扩容和搬迁变得危险。 让用户保存醒体数组下标，也会使睡眠和删除改变对象身份。

当前设计把身份与存储位置分开：

1. 用户持有 `b2BodyId`。
2. `index1 - 1` 对应世界中的刚体身份槽。
3. `generation` 用于识别槽位重新使用。
4. `b2Body.setIndex` 指向当前 solver set。
5. `b2Body.localIndex` 指向该集合里的稠密位置。

因此一个刚体可以换集合、换稠密下标，公开 ID 仍然不变。 稳定的是身份，不是热数据地址。

手算 RemoveSwap

设某稠密数组为：

下标	对象
0	A，ID 10
1	B，ID 20
2	C，ID 30
3	D，ID 40

删除 B 时，不把 C、D 全部向前移动。 而是把尾部 D 复制到下标 1，再把有效数量从 4 改成 3：

下标	删除后对象
0	A，ID 10
1	D，ID 40
2	C，ID 30

必须同时把 D 的 `localIndex` 从 3 改成 1。 D 的公开 ID 40 没有变。

`b2RemoveBodySim` 专门处理这项映射修复。 一般数组的 `b2Array_RemoveSwap` 返回被交换元素原来的尾下标，供调用者更新其反向索引。 如果删除的本来就是尾元素，则返回 `B2_NULL_INDEX`。

重要不变量：醒体的 `bodySims[i]` 和 `bodyStates[i]` 必须代表同一个刚体。 因此删除醒体时，两份数组必须在同一位置执行相同的尾交换。 只搬 Sim 不搬 State，会把一个刚体的质量和另一个刚体的速度配在一起。

代价：RemoveSwap 不保持原遍历顺序。 所有依赖该顺序的算法，都必须把确定性和索引修复一起考虑。 数组增长也仍会使旧指针失效。

实验：用四个物体重复删除中间和末尾元素。 保留公开 ID 检查身份，用内部断言检查 Sim、State 和 `localIndex` 的一致性。

18.4 SolverSet 把“不参与热循环”落实到布局

仅有一个 `sleeping` 布尔值，不能阻止求解器扫到睡眠物体。 当前 solver set 将它们从热数组中分离。

集合编号	含义
0	`b2_staticSet`
1	`b2_disabledSet`
2	`b2_awakeSet`
3 及以上	睡眠集合

其中有两个不能省略的区别。

第一，`disabledSet` 不只存用户禁用的刚体和关节。 它还承接某些未触碰接触，例如初次创建接触时两端都不在 awake 集合。

第二，awake 集合中的 `contactSims` 是未触碰接触。 正在触碰的接触和醒着的关节放在约束图对应颜色中。 不能说“awakeSet 包含所有醒体约束”。

`bodyStates` 只存在于 awake 集合，用于醒着的动态和运动学刚体。 静态体没有这份可写状态。

睡眠集合通常只有一个岛，但这不是永久一一对应关系。 在睡眠岛之间创建关节可能合并睡眠集合，使一个集合暂时拥有多个岛。 `solver_set.h` 和唤醒函数明确保留了这种情况。

收益：积分与约束热循环处理的是醒体工作集，不必反复过滤全世界。 **代价：**进入和离开集合要复制数据、修映射，有时还要扩容。 因此一次迁移不是常数时间修改标志。

18.5 岛是睡眠连通分量，不是 SIMD 工作块

如果每个刚体独立睡眠，可能出现一个箱子刚被暂停，支撑它的另一个箱子却还在移动。 接触约束连接的状态不能随意只停一半。

岛把通过触碰接触和关节连接的非静态刚体组织起来，作为睡眠判断单位。 一个成员还不满足睡眠条件，就可以让整岛保持醒着。

静态地面不把所有落在地面的箱子合并成一个大岛。 否则世界里一个箱子移动，就可能阻止全部堆叠睡眠。

运动学体与静态体在这里不能混称。 当前拆岛逻辑连接非静态端，运动学体可以参与岛连接。 这与下一章“图着色只给动态写端占位”的判据不同。

Box2D 的求解并行主要按约束图颜色组织。 岛在本章首先回答“谁一起睡、谁一起醒”，不应套用旧版本按岛划分求解工作的解释。

18.6 新连接合并，删除连接延后检查

最朴素的连通分量维护方式，是每帧重新遍历全部刚体和约束。 这容易理解，却会重复检查大量没有变化的连接。

当前合并路径在出现连接时增量处理。 `b2MergeIslands` 按刚体数量保留大岛，把小岛成员迁入大岛。

具体步骤为：

1. 两端本来同岛则返回。
2. 静态端没有岛时，保留另一端岛。
3. 比较两个岛的刚体数量，选择大岛。
4. 迁移小岛的 body、contact、joint 列表。
5. 修正每个成员的 `islandId` 和 `islandIndex`。
6. 累加待检查的约束删除计数，销毁小岛。

例如 1 体岛连接 10,000 体岛，改小岛一侧通常比搬大岛便宜。 但连接列表也需要迁移，因此不能只按刚体数宣称总成本恰好为一次赋值。

删除连接更麻烦：删掉一条边，不一定真的切断了图。

- A—B—C—D 删除 B—C，会得到两块。
- 若还存在 A—D，删除 B—C 后仍然连通。

所以 `b2UnlinkContact`、`b2UnlinkJoint` 增加 `constraintRemoveCount`，先保守保留原岛。 等岛中出现足够困倦的成员，再考虑拆分。

收益：避免每次接触短暂消失都立即做完整连通分析。 **代价：**已经断开的部分可能暂时还共享睡眠单位，延迟进入睡眠。

18.7 当前拆岛用临时并查集

不要把当前 `b2SplitIsland` 描述成沿刚体接触链表做一次旧式 DFS。 它对该岛现有成员建立临时并查集。

主要过程为：

1. 为每个原岛刚体建立 `parent` 与 `rank`。
2. 遍历仍存在的接触，把两个非静态端 union。
3. 遍历关节，执行同样的连接合并。
4. 对根节点统计组件数量和各组件约束数量。
5. 若仍只有一组件，清除删除计数后返回。
6. 否则预先计算新岛容量，再创建新岛。
7. 分配 body、contact、joint 到各新岛，修复索引。
8. 销毁旧岛并逆序释放临时数组。

对 A—B 与 C—D 两条剩余边，union 后得到 $\{A, B\}$ 和 $\{C, D\}$。 若加上 A—D，所有 parent 最终归于一个组件，不需要真正搬家。

令 V 为岛内刚体数，E 为接触与关节数量。 并查集部分的常见摊还复杂度写作：

$$O((V + E)\alpha(V)),$$

其中 α 为反阿克曼函数，在实际规模中增长极慢。 另外还有成员计数、数组分配和迁移成本，不能忽略。

拆岛任务不能与 `FinalizeBodies` 并发修改岛索引。 当前调度在安全区间发出一个拆岛任务，并在 Finalize 前等待完成。 每个 worker 推荐最困的候选；睡眠时间相等时，较大的岛 ID 优先，以保持确定性。 跨 worker 归约也使用同一平局规则。

实验：构造链条，删除桥边与非桥边各一次。 记录组件数量、删除计数和何时真正拆岛，验证“删除边”与“出现新岛”不是同一事件。

18.8 睡眠不能只检查线速度

一个细长物体质心几乎不动，但可能还在快速旋转。 一个堆叠速度接近零，也可能持续依赖位置修正。 只看 $|v|$ 会过早宣布它稳定。

当前 Finalize 使用以下变量：

符号	含义	单位
v	质心线速度	m/s
ω	角速度	rad/s
R	`maxExtent`	m
Δp	大步累计增量位移	m
$\Delta q.s$	增量旋转的正弦分量	无量纲
Δt	整个大步时间	s

代码先估计最远点速度和大步运动量：

$$u = |v| + |\omega| R,$$

$$d = |\Delta p| + |\Delta q.s| R.$$

再定义睡眠指标：

$$s = \max\left(u, \frac{0.5 d}{\Delta t}\right).$$

s 的单位是 m/s。 这里取最大值，不是把两项直接相加。 分母是大步 Δt，不是某个子步的 h。 $\Delta q.s$ 是正弦分量，不是直接保存的旋转角。

数值例：速度很小，仍然不能睡

设：

- $|v| = 0.01\ \mathrm{m/s}$。
- $|\omega| = 0.02\ \mathrm{rad/s}$，$R = 0.5\ \mathrm{m}$。
- $|\Delta p| = 0.001\ \mathrm{m}$，$|\Delta q.s| = 0.002$。
- $\Delta t = 1/60\ \mathrm{s}$。

于是：

$$u = 0.01 + 0.02 \times 0.5 = 0.02\ \mathrm{m/s},$$

$$d = 0.001 + 0.002 \times 0.5 = 0.002\ \mathrm{m},$$

$$s = \max(0.02, 0.5 \times 60 \times 0.002) = 0.06\ \mathrm{m/s}.$$

默认睡眠阈值为 0.05 m/s，因此该物体的睡眠计时清零。 真实速度虽然不大，本步运动指标却仍然过大。

若改成 $|\Delta p| = 0.0002\ \mathrm{m}$、$|\Delta q.s| = 0.0004$，则 $d = 0.0004\ \mathrm{m}$。 位置项为 0.012 m/s，最终 $s = 0.02\ \mathrm{m/s}$，可以累计睡眠时间。

边界条件

`b2FinalizeBodiesTask` 在以下任一条件满足时清零 `sleepTime`：

- 世界禁用睡眠。
- 刚体禁用睡眠。
- `sleepVelocity > sleepThreshold`。

注意比较为严格大于，所以等于阈值时可以累计。 满足安静条件则 `sleepTime += timeStep`。 默认 `B2_TIME_TO_SLEEP` 为 0.5 s。

理想算术下，60 Hz 的 30 帧正好是 0.5 s。 实际精确睡眠步号还受浮点累加、运动变化和拆岛等待影响，不能保证任何场景都在第 30 帧睡眠。

若岛内任意体的 `sleepTime < 0.5`，该 worker 就设置岛的 awake 位。 所有 worker 的位图按位或以后，只要还有一个成员要求清醒，整岛就不能入睡。

18.9 入睡时真正发生的数据迁移

`b2TrySleepIsland` 首先拒绝“仍待拆分且成员数大于一”的岛。 单体岛是这个条件的例外，不应省略后半个判断。

之后不是只设置一个布尔标志，而是执行搬迁：

1. 分配睡眠集合，并按岛大小预留数组容量。
2. 将 body Sim 从 awake 复制到 sleep。
3. 从 awake Sim 数组 RemoveSwap，并修正被交换体索引。
4. 从 awake State 数组同位置 RemoveSwap；不复制睡前 State。
5. 更新原体的 `setIndex/localIndex` 和睡眠事件。
6. 把触碰 contact Sim 从约束图移到睡眠集合。
7. 清除正常颜色中的占位 bit，并修复交换接触索引。
8. 迁移关节与岛描述。

未触碰接触有额外所有权判断。 如果另一端仍然 awake，这个接触继续保留在 awake 集合。 只有另一端也不 awake 时，才把它移入 disabled 集合。

原因是未触碰接触可能跨越不同的睡眠岛，没有明确的单岛归属。 不能简单说“睡眠岛的全部接触都进入它的睡眠集合”。

睡眠阶段放在最后，还因为它会打乱 awake 稠密索引。 如果在 enlarged body 位图消费之前搬家，位图中的下标就可能指向别的体。 当前倒序枚举待睡岛，也是在配合 RemoveSwap 的行为。

18.10 唤醒不是恢复一份隐藏速度

唤醒反向迁移 Sim，但并不恢复睡前 State。 源码非常明确：

```
b2BodySim* simDst = b2Array_Emplace( awakeSet->bodySims );
memcpy( simDst, simSrc, sizeof( b2BodySim ) );

b2BodyState* state = b2Array_Emplace( awakeSet->bodyStates );
*state = b2_identityBodyState;
state->flags = body->flags;
```

新状态的线速度、角速度和增量位移为零。 增量旋转为 $(1, 0)$，即单位旋转，而不是全零。

因此“从 BodySim 取出速度重建 State”是错误解释：BodySim 本来就没有这些速度字段。 唤醒后再施加冲量或设置速度，是另一项操作。

接下来恢复未触碰接触、重新把触碰接触与关节加入约束图，并迁回岛描述。 接触需要重新染色，因此唤醒不是免费操作。

内存收益的精确说法

睡眠减少 `bodyStates.count`，缩小有效热工作集。 但 RemoveSwap 不会自动缩减数组 `capacity`。 不能说每睡一个体，就立刻向系统归还 32 B。

这里主要避免的是反复读取和求解睡眠体。 频繁醒睡反而可能增加复制、分配和重新染色成本。

实验：让两个不相连箱堆落在同一静态地面上。 都睡眠后只推动其中一堆，检查另一堆是否保持睡眠，并记录 awake 数量和集合迁移。

18.11 临时栈分配也有真实代价

宽约束、工作块和临时索引数组通常只活过一个步骤或阶段。 朴素方式每次调用通用分配器；稳定重复的高峰则适合集中管理。

当前 `b2StackAlloc` 对正请求大小按 32 B 向上对齐。 容量内使用 bump index，容量不足则回退到堆分配。 每一笔分配仍会记录一项 entry。

设容量为 128 B，依次请求 1、33、64 B：

请求	对齐后大小	处理
1	32	栈内，累计索引 32
33	64	栈内，累计索引 96
64	64	超容量，堆回退

总分配高峰为 $32 + 64 + 64 = 160$ B。 释放必须按 64、33、1 的逆序进行。 堆回退的那笔由通用释放处理，其他笔回退索引。

所有分配释放后，`b2GrowStack` 才允许增长。 此例下一次容量可增长到：

$$160 + 160/2 = 240 \text{ B}.$$

下一轮同样请求就不需要那次堆回退。 但这不等于“每帧一个 O(1) reset，任何时候都零分配”。 entry 数组可能增长，每笔仍有登记和释放，首次高峰也可能走堆。

实验：区分首帧与稳定期，记录 capacity、max allocation 和回退次数。 不要删除首帧成本后，又把稳定期数字解释为整个生命周期从不分配。

18.12 自测与参考答案

1. 删除稠密数组第二项时，最后一项的公开 ID 会改变吗?
2. 1,000 个有效 `b2BodyState` 占多少有效数据字节?
3. 上述第一组睡眠例中最终指标是多少?
4. 两个箱堆共享静态地面，是否因此必然同岛?
5. 唤醒的初始速度从哪里恢复?
6. 一次 RemoveSwap 是否保证释放底层容量?
7. 128 B 栈容量下请求 1、33、64 B，有几次堆回退?

参考答案：

1. 不变，改变的是 `localIndex`，并需修复映射。
2. 32,000 B，不包括保留容量、其他数组和分配器元数据。
3. 0.06 m/s，大于默认 0.05 m/s，计时清零。
4. 不必；静态地面不桥接它们的睡眠连接。
5. 不恢复睡前速度，新建 identity State；后续输入另行改变速度。
6. 不保证，它只减少有效数量。
7. 一次；对齐后的总请求高峰为 160 B。

18.13 源码导航

主题	相对源码链接与入口行号
Body、State、Sim	`body.h`，68、153、175
Sim 删除修索引	`body.c`，`b2RemoveBodySim`：34
通用 RemoveSwap	`container.h`，136
集合定义及多岛情况	`solver_set.h`，14、54
唤醒与入睡迁移	`solver_set.c`，35、155

主题	相对源码链接与入口行号
初始接触集合选择	`contact.c`，247–258
岛合并、删除与拆分	`island.c`，82、235、311、389
睡眠指标	`solver.c`，`b2FinalizeBodiesTask`：560
拆岛调度与最终睡眠	`solver.c`，1454、2000
栈分配与增长	`arena_allocator.c`，30、65、83

返回 README · 下一章：图着色、SIMD 与多线程